个人总结

高并发

可以直接这样回答，控制在 **1 分钟左右**：

> Java 解决高并发，我主要从四个方面考虑：
>
> **第一，提高性能。** 用缓存减少数据库访问，优化慢 SQL，合理配置线程池，减少锁竞争。
>
> **第二，分担压力。** 用消息队列异步处理、削峰填谷，通过负载均衡和多实例部署提高处理能力。
>
> **第三，保护系统。** 用限流控制请求量，避免系统过载；下游持续异常时通过熔断暂停调用；资源紧张时降级非核心功能，优先保障核心业务，同时做好超时和资源隔离。
>
> **第四，保证数据正确。** 单机用锁或原子类保证线程安全，多实例下通过数据库条件更新、唯一约束和事务等防止超卖、重复下单，并做好幂等。
>
> 具体采用哪些措施，要先通过压测找到瓶颈，再验证优化效果。

记住四句话：**提高性能、分担压力、保护系统、保证正确。**

> > 服务雪崩是指一个服务失败，导致整条链路的服务都失败的情形，一般我们在项目解决的话就是两种方案，第一个是服务降级，第二个是服务熔断，如果流量太大的话，可以考虑限流
> >
> > 服务降级：服务自我保护的一种方式，或者保护下游服务的一种方式，用于确保服务不会受请求突增影响变得不可用，确保服务不会崩溃，一般在实际开发中与feign接口整合，编写降级逻辑
> >
> > 服务熔断：默认关闭，需要手动打开，如果检测到 10 秒内请求的失败率超过 50%，就触发熔断机制。之后每隔 5 秒重新尝试请求微服务，如果微服务不能响应，继续走熔断机制。如果微服务可达，则关闭熔断机制，恢复正常请求

左右连接和内连接

连接方式	返回哪些数据	未匹配的部分
`INNER JOIN`（内连接）	只返回两表满足连接条件的行	不保留
`LEFT JOIN`（左连接）	保留左表全部行，匹配右表数据	右表字段填 `NULL`
`RIGHT JOIN`（右连接）	保留右表全部行，匹配左表数据	左表字段填 `NULL`

冒泡排序和二分查找

冒泡排序： 从头开始比较相邻两个数，前面的比后面的大就交换。一轮结束，最大的数就到了最后。接着对剩下的数重复这个过程，直到排好序。

二分查找： 前提是数组有序。每次看中间的数，目标比它大就找右半边，比它小就找左半边。不断缩小范围，直到找到，或者范围为空。我的问题

LangSmith

LangSmith 是一个大模型应用的可观测性与评测平台，通过追踪模型、检索和工具调用的完整链路，帮助开发者调试问题、评估回答质量，并优化性能与成本。

> LangSmith是帮助开发者一站式完成对复杂 AI 应用的调试、测试、评估与生产环境监控的一个平台

> OpenSpec 的作用是让 AI 在写代码前先明确需求、设计和任务清单，再按规范实施与验证，减少理解偏差和返工。

命令	用途
`/opsx:explore`	探索需求、分析代码、讨论方案
`/opsx:propose`	创建变更，一次生成提案、规格、设计和任务
`/opsx:apply`	按任务清单实现代码
`/opsx:update`	修改变更的规划文档，保持内容一致
`/opsx:sync`	将变更中的规格合并到主规格
`/opsx:archive`	归档已完成的变更

skill和prompt以及mcp的区别是什么?

> “**MCP** 负责在底层把工具标准化接进来，**Prompt** 负责用文字把业务步骤讲清楚，而 **Skill** 就是把**‘某项业务的专属 Prompt’和‘要用到的几个 MCP 工具’打包在一起**，做成一个按需加载的业务模块。”

TEXT2SQL

在我们的项目里，核心业务库大概有 30 张表。在做 Text2SQL 的时候，我们肯定不敢让大模型直接裸写 SQL 就往数据库里跑，而是把整个流程分成了三步来把关： **生成前**，先用检索挑出相关的几张表，连同业务计算口径一起喂给模型； **生成后**，用 SQL 解析器做一道语法和白名单校验，报错了就让它自动重试； **最后执行时**，全部走只读从库，加上超时和行数限制，确保绝对不会把线上数据库拖垮。”

分布式

“AT 是 Seata 基于两阶段提交实现的分布式事务模式：一阶段将业务数据和回滚日志一起提交，二阶段成功就清理日志，失败就根据日志自动补偿回滚。”（官方说明）

eventlop

事件循环的执行顺序大致如下：

1. 从主线程中取出一个同步任务放入调用栈中执行，直至调用栈为空。
2. 检查微任务队列，如果存在微任务，则依次执行完微任务队列中的所有任务。
3. 检查宏任务队列，选择队列中的第一个任务放入调用栈执行。
4. 重复上述步骤。

RBAC

我们的RABC是使用的七张表

> - 7张表（用户表、角色表、权限表、菜单表、用户角色中间表、角色权限中间表、权限菜单中间表）
>
> - 张三登录系统---> 查询张三拥有的角色列表--->再根据角色查询拥有的权限 ->

再根据权限查询拥有的菜单

- 整体都多对多的关系

LangGraph 通过 **Checkpointer** 实现状态持久化。编译图时配置 Checkpointer，执行时传入 **thread_id** 标识会话或工作流，框架会按执行轮次保存 Checkpoint，包括 State 和后续执行信息。开发可以用内存实现，生产通常使用 Postgres 等持久化后端，保证进程重启后状态仍然保留。

断点续跑时，使用相同的 thread_id，通过 `graph.invoke(None, config)` 读取检查点并继续执行，不需要从头跑完整个流程。如果是故障恢复，需要先排除故障；如果要设置静态断点，可以配置 `interrupt_before` 或 `interrupt_after`。

人工介入通过 `interrupt(payload)` 实现。在需要审批的节点调用它，流程暂停，并把待审核信息返回给应用。人工处理后，用相同的 thread_id 调用 `graph.invoke(Command(resume=审批结果), config)`，审批结果会成为 `interrupt()` 的返回值，节点再根据结果继续执行。

需要注意，人工恢复时，包含 interrupt 的节点会从头执行，前面的代码可能重复运行，所以外部写操作要保证幂等，避免重复发邮件或重复创建订单。

是的，配置 Checkpointer 就是在启用 LangGraph 的线程级短期记忆。 更准确地说，它负责保存和恢复图的 `State`，对话历史只是 State 的一部分，所以截图里也用它实现断点恢复和人工介入。官方文档

可以这样理解：

- **State**：记什么，例如聊天记录、工具结果、业务状态。
- **Checkpointer**：负责保存、恢复这些状态。
- **thread_id**：标识哪一个会话，多次调用使用相同 ID，就能接着已有状态运行。

这里的"短期"指作用范围是同一个会话，不是保存时间短。 即使用 PostgreSQL 保存，重启后还能恢复，它仍属于短期记忆；跨不同会话共享用户偏好、个人信息等，通常使用 **Store 实现长期记忆。**

那长期记忆呢?

长期记忆通常用 `Store` 实现，用来跨会话保存和读取信息，例如用户偏好、个人资料、历史经验。官方文档

对比	短期记忆	长期记忆
组件	`Checkpointer`	`Store`
范围	同一个 `thread_id`	可跨不同 `thread_id`
典型内容	当前聊天记录、执行状态	用户偏好、长期事实
数据组织	按线程保存状态	自定义 namespace，常包含 `user_id`

比如你在会话 A 说"我是 Java 开发，回答请用中文"，应用把这些信息写入 Store。以后开启会话 B，虽然 `thread_id` 变了，仍可以通过同一个 `user_id` 读取。

配置和存取示意如下，假设图的 `builder` 已定义：

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()
graph = builder.compile(checkpointer=checkpointer, store=store)

# 用用户 ID 组织记忆，供该用户的不同会话使用
namespace = ("users", "user_123", "memories")

store.put(namespace, "profile", {
    "job": "Java开发",
    "language": "中文",
})

memories = store.search(namespace)
```

配置 Store 后，还需要应用决定记什么、何时写入，以及读取后怎样放进模型上下文。 它不会自动把所有聊天转成长期记忆。上面的 `InMemoryStore` 适合演示，进程重启会丢失；需要持久保存时可使用 `PostgresStore` 等后端。

面试官问我，我项目里的电商系统从架构方面是如何设计的，我该如何回答

明白，那你需要准备的是 **Spring Cloud 技术栈下的架构回答**。重点是把服务如何划分、请求怎么流转、关键问题怎么处理讲清楚，并与简历中的组件保持一致。

下面是一版参考口径，其中网关、缓存、消息队列等细节，要对应到你实际实现或能展示的改造方案。

你可以这样回答：

> 我们的电商系统整体采用基于 Spring Cloud 的微服务架构，主要分为接入层、业务服务层和数据存储层，围绕商品浏览、下单支付以及会员运营展开。
>
> **接入层通过网关统一承接请求**，主要处理路由转发、登录身份校验和限流等公共逻辑，然后把请求转发到对应的业务服务。涉及订单归属、会员权益等具体权限，仍然由业务服务校验。
>
> **业务层按照业务职责划分服务**，主要包括商品、会员、订单、营销和支付。商品服务负责商品资料、分类和详情；会员服务负责用户资料、收货地址和会员权益；营销服务负责优惠券及活动规则；订单服务负责创建订单和订单状态管理；支付服务负责支付渠道接入及支付结果处理。库存是否独立拆分，则根据实际业务复杂度决定。
>
> 这样划分主要是为了明确业务和数据归属。例如商品展示调整集中在商品侧，支付渠道变化集中在支付侧；商品查询流量较大时，也方便针对商品服务扩容。不过拆分之后也增加了远程调用、故障处理和数据一致性的复杂度。
>
> **服务之间根据调用目的选择通信方式。** 下单时必须立即得到结果的操作，比如商品校验、优惠计算和库存确认，使用同步接口调用；支付成功后的通知、积分发放等允许稍后完成的操作，如果项目接入了消息队列，就通过事件异步处理，缩短核心交易链路。
>
> **数据方面强调各服务管理自己的业务数据。** 其他服务通过接口获取数据，避免直接修改别人的业务表。商品详情这类读取频繁的数据可以增加缓存；创建订单时，则需要重新校验价格、优惠和库存，并在订单中保存成交时的商品及价格快照。

> 这个架构中，我重点关注的是交易链路的正确性，包括重复下单、库存并发、支付重复回调，以及跨服务调用失败后的恢复。我的个人工作主要是……这里再接自己负责的模块和具体实现。

面试时，**最好再用一个下单例子把这些服务串起来**：

> 用户提交订单后，请求先经过网关进入订单服务。订单服务校验请求，获取商品信息、计算优惠，并协调库存预占，然后保存订单及明细，返回待支付订单。用户发起支付时，由支付服务创建支付请求；支付结果经过验签、金额核对和幂等处理后，再驱动订单进入已支付状态。超时未支付的订单需要关闭，并协调释放库存及优惠券占用。

这里的"库存预占、优惠券占用、超时关闭"属于具体方案，只有与你的实现一致时才这样讲。

你接下来最需要统一的是这些容易被追问的细节：

架构点	需要能回答的问题
服务划分	实际拆了哪些服务？库存、积分分别归谁管理？
Spring Cloud 组件	网关、注册配置中心、服务调用分别用了什么？各自解决什么问题？
数据归属	独立库、独立 schema，还是共享数据库？是否存在跨服务查表？
下单一致性	库存预占成功但订单保存失败，如何释放库存？
支付可靠性	回调重复、回调丢失、金额不一致分别怎么处理？
异步处理	消息发送失败、重复消费、消费失败如何恢复？

"使用 Spring Cloud"只是技术背景，真正体现架构理解的是：为什么这样拆、失败时怎么办、你具体做了哪一部分。 原项目事实和 Java 改造设计也要分清；如果属于改造或复现，直接说明，更容易把后续追问讲扎实。

如果与你实际分工一致，最后接一句：

> 我主要负责商城端的交易流程和接口对接，包括结算、支付入口、特殊订单分支，以及会员和积分兑换交互。我的重点是把多个服务的接口串成完整流程，并处理参数校验、业务分支和失败提示。

幂等性

我采用乐观锁思想，通过订单号和预期状态做原子条件更新，只有更新成功的请求才执行业务；再通过本地事务保证状态更新与业务操作的一致性，实现支付回调的幂等。

> 我通过带状态条件的原子 UPDATE 实现幂等控制，只有影响行数为 1 的请求才执行业务，重复回调不会再次执行业务。这种方式类似 CAS，采用了乐观锁思想。

幂等性实操

方案一：数据库唯一索引（防重复插入最简单）

- **场景**：防止重复创建订单、重复注册。
- **做法**：给数据库的某个业务字段加唯一索引（Unique Index），比如"订单流水号 `order_no`"。
- **效果**：第 1 次请求 `INSERT` 成功；第 2 次相同的请求进来，数据库直接报错 `DuplicateKeyException`（唯一键冲突），Java 捕获这个异常后直接返回"已处理成功"，就不会插入两条数据了。

方案四：状态机约束（利用业务本身的状态流转）

- **场景**：订单支付、发货、退款等有固定顺序的业务。
- **做法**：订单状态只能从 `1（待支付）` -> `2（已支付）` -> `3（已发货）`。
    - 当处理“支付成功”回调时，SQL 这么写：

      SQL

      ```
      UPDATE orders SET status = 2 WHERE order_no = 'XYZ' AND status = 1
      ```

- **效果**：如果微信/支付宝重复发了两次“支付成功通知”，第 1 次把状态改成了 `2`，第 2 次再来时，`status = 1` 的条件已经不满足了，自然什么都不会发生。

一句话总结： Java 里的幂等性，其实就是**给代码加一层“防重过滤网”**（靠唯一单号、Redis Token、版本号或状态判断），保证同一个请求哪怕因为网络卡顿或用户手抖来了好几次，系统也只认第一次的账。

> **那方案四的话还需要加分布式锁吗?**
>
> 取决于你的具体业务流程：**如果仅仅是单纯地改一下数据库状态，不需要加分布式锁；但如果改状态前有“前置耗时操作”，或者需要防恶意高频重试，就必须加分布式锁。**
>
> 数据库本身的 `UPDATE ... WHERE status = 1` 自带**行级排他锁**，在数据库层面已经是并发安全的了。之所以很多大厂代码里既用了状态机（方案四），又加了 Redis 分布式锁，主要是为了解决以下 **3 个单靠数据库搞不定的问题**：
>
> ## 1. 什么情况下【不需要】加分布式锁?
>
> 如果你的接口逻辑非常纯粹，**进来第一件事就是执行那条 `UPDATE` 语句**，且后续操作全由这条 SQL 的返回值（`rows == 1`）来控制，那么**完全不需要加分布式锁。**
>
> Java

```
@Transactional
public String handlePayCallback(String orderNo) {
    // 进来直接撞数据库，利用 MySQL 行锁抢执行权
    int rows = orderMapper.updateStatus(orderNo, 1, 2);

    if (rows == 0) {
        // 没抢到，说明别的线程改过了，直接退出
        return "SUCCESS";
    }

    // 抢到了 (rows == 1)，安全地执行后续本地数据库操作
    pointMapper.addPoints(userId, 100);
    return "SUCCESS";
}
```

原因：没有前置废话，MySQL 的行锁已经帮你完成了“谁先抢到谁执行”的并发控制，再加一层 Redis 分布式锁纯属画蛇添足。

2. 什么情况下【必须】加分布式锁?

一旦你的业务出现下面这 3 种情况，单靠方案四就会出漏洞或性能瓶颈，必须配合分布式锁：

情况 A：在 `UPDATE` 之前，有复杂的校验或第三方接口调用（最常见！）

很多业务在真正改数据库状态前，需要先做一堆前置动作（比如查订单详情、计算优惠券、调风控接口、调银行接口确认账单）。

看看如果不加分布式锁会发生什么：

Java

```
public String refundOrder(String orderNo) {
    // 1. 先查订单
    Order order = orderMapper.selectByOrderNo(orderNo);
    if (order.getStatus() != 2) return "不可退款";

    // ⚠️ 漏洞区：如果两个并发请求同时走到这里，此时数据库状态还没改（还是 2）！
    // 它们会同时调用微信退款接口，导致调了两次微信 API！
    wechatPayService.doRefund(orderNo);

    // 2. 最后才用方案四改状态（太晚了，钱已经退过两次了）
    int rows = orderMapper.updateStatus(orderNo, 2, 3);
}
```

- **为什么方案四救不了?** 因为方案四只在执行 `UPDATE` 的那一瞬间加锁，管不住 `UPDATE` **之前**的代码。
- **怎么解决?** 在方法最开头加上 Redis 分布式锁（`lock(orderNo)`），保证同一时刻只有一个线程能进入整段逻辑。

情况 B：防止“无效重复请求”浪费数据库连接（保护 MySQL）

- **Redis 扛并发能力**：单机轻松扛 5万~10万 QPS。
- **MySQL 扛并发能力**：普通配置也就几千 QPS，而且数据库连接（Connection）是非常昂贵的资源。

- 如果前端有个 Bug，或者上游服务抽风，1 秒内发了 500 个相同的请求过来。如果不加 Redis 锁，这 500 个请求会全部穿透到 MySQL 去抢行锁、开启事务，白白消耗数据库连接。加了 Redis 锁后，后 499 个请求在耗时不到 1 毫秒的内存层就被直接打回了。

情况 C：状态流转存在“ABAB”反复横跳（ABA 问题）

方案四天然防不住状态“绕一圈又回来”的情况。

- 比如某个业务状态是：`1（未开启）` ↔ `2（已开启）` 可以来回切换。
- 线程 A 和线程 B 都是要把状态从 `1` 改成 `2`。如果线程 A 改成了 `2`，接着用户又操作把状态改回了 `1`，这时候卡住的线程 B 终于执行了，它一看数据库里是 `1`，就会错误地再次改成 `2`。这种场景需要分布式锁或版本号（方案三）来配合。

3. 一张表帮你做决定：到底加不加？

你的业务特征	是否需要分布式锁？	推荐方案
简单直接：进来第一行代码就能直接跑 `UPDATE ... WHERE status = X`，且单号不会遭遇恶意高频轰炸	**不需要**	单纯用 **方案四（数据库状态机）** 即可，简单高效。
先查后改 / 调外部接口：在 `UPDATE` 之前，必须先查询数据做复杂校验，或者要先调用第三方 HTTP/RPC 接口	**必须加！**	**Redis 分布式锁 + 方案四**（锁保护整个方法，状态机做最终数据兜底）。
高危核心资金链路：涉及提现、退款、大促秒杀，要求极致的数据库保护	**强烈建议加**	**Redis 分布式锁 + 方案四**（Redis 挡流量，MySQL 保账目）。

💡 补充一个经典踩坑点：如果两者一起用，顺序千万别写反！

如果你决定用 **“分布式锁 + 方案四”**，并且方法上有 Spring 的 `@Transactional` 事务注解，**锁的范围必须大于事务的范围**：

- ❌ **错误写法**：在方法上加 `@Transactional`，在方法内部加 `lock()` 和 `unlock()`。
    - **后果**：锁都已经释放了（`unlock`），但 Spring 事务还没提交（`commit`）。此时第二个并发线程抢到锁进来，读到的还是未提交的旧状态，照样出现并发 Bug。
- ✅ **正确做法**：**先加锁** → **开启事务执行 SQL** → **提交事务** → **最后释放锁**（通常把加锁逻辑写在 Controller 层或 AOP 切面里，去包裹住带 `@Transactional` 的 Service 方法）。

面试官问，商城里面有百万商城，建表的时候如何操作

再给你浓缩成 **30 秒就能顺下来**的极简大白话版本，记住“1个前提 + 3个动作”就行：

直接这么说：

“百万级数据对 MySQL 单表其实不算大，不用上来就搞分库分表。建表时核心就是**别搞大宽表**，我主要做三件事：

1. **把表拆成四份（冷热、动静分离）**：
    - **商品主表（SPU）**：只放商品名、分类、状态这些短字段，保证列表查得快。
    - **详情表**：图文介绍那种大文本单独拆出去，点进详情页才查，不占数据库缓存。
    - **规格表（SKU）**：存具体的价格，还有用 **JSON 字段**存颜色、尺码这些动态属性，避免表里建一堆空字段。
    - **库存表**：单独拆出来。因为商品信息是'光读不写'，库存是'高频修改'，拆开是为了防止下单扣库存锁表时，把正常看商品的人给卡住。
2. **抠一下字段和索引细节**： 价格用 `DECIMAL` 或直接存'分'（整数），字段全设 `NOT NULL`。按高频查询（比如店铺+分类+状态）建好联合索引。另外表里都带个 `shop_id`（店铺 ID），万一以后数据破千万要分库分表，直接拿它当分片键。
3. **别让 MySQL 硬扛搜索**： 建表只是存底账。真正线上的多条件筛选和模糊搜索，直接通过 Binlog 把数据同步到 **ES** 里去查，热门商品和库存扔 **Redis**，数据库基本就没压力了。"

支付补偿机制

面试时可以直接这样说，语气自然一些：

"我会加一个主动查单的补偿机制，不会一直等回调。

用户发起支付时，我们本地会先记一笔支付流水。如果过了一段时间，这笔支付还没拿到明确结果，定时任务就拿商户支付单号去支付平台查询。

如果查到已经支付成功，就补上本地的支付状态，再通知订单系统继续发货或者发放权益。如果查单超时了，或者平台还在处理中，就隔一段时间再查，**不能因为没收到通知就当作支付失败。**多次查不到明确结果的，会记录下来并告警，继续通过对账或人工核实。

这里还有一个关键点：**补偿任务和支付回调可能同时到达，所以必须做幂等。** 两边走同一套支付成功处理逻辑，用数据库条件更新控制状态只变更一次。后面的发货消息可以通过本地消息表保证可靠发送，消费端也要防止重复发货。

最后再加一层定期对账，把平台账单和本地流水核对一下，找出漏处理的支付。如果发现钱已经付了，但订单已经关闭、也没法履约，就走退款流程，并跟踪退款结果。"

记住这条主线就好：**主动查单确认结果，幂等补单推进业务，定期对账兜底。**

高并发券

去掉 Nginx 后，整体架构更聚焦在 Java 微服务体系上，逻辑反而更紧凑。你可以直接按下面这套"四层漏斗"话术来回答，既有技术细节，又非常贴合实战：

💡 面试直接背诵版（口语化、紧凑）：

"面对10万用户抢1000张券的场景，99%都是无效请求，我的核心思路是做'**漏斗式层层拦截**'，把绝大部分流量挡在数据库前面，具体分四层来做：

第一层是前端拦截： 用户点击抢购后，按钮立马**置灰防抖**，禁止重复点击；同时在开抢前加一道**滑块验证码**，既能挡住一部分脚本机器，又能把10万人在同一毫秒的并发打散、平摊到几秒钟内。

第二层是网关层限流（Spring Cloud Gateway + Sentinel）：

> 请求到了网关，我主要做两道拦截：
>
> 1. **防刷限流：** 提取客户端 **IP 或登录 Token 里的 UserID** 作为限流 Key，限制单用户/单IP每秒最多请求 2~3 次，直接把黄牛和连点器挡在外面。
> 2. **接口全局限流：** 用**令牌桶算法**控制总流量。因为只有1000张券，网关最多只放行比如 3000 个请求去往后端微服务，剩下的 9 万多请求直接在网关触发降级，快速返回'当前抢购人数过多，请稍后再试'。
>
> **第三层是业务服务层（纯内存操作）：** 放行进来的这 3000 个请求，绝对不能直接查数据库。我们会提前把 1000 张券的库存预热到 **Redis** 中，用 **Lua 脚本**做原子性的'判重+扣减库存'，保证不超卖、不重复领。一旦 Redis 库存扣到 0，立刻在当前微服务的 **JVM 本地内存**打个'已售罄'的布尔标记，后续再漏进来的请求连 Redis 都不用查，直接看本地内存就打回。
>
> **第四层是数据库层（异步削峰）：** 真正从 Redis 里抢到券的那 1000 个请求，直接发一条消息到 **MQ（如 RocketMQ）**，前端直接提示用户'抢购成功，优惠券发放中'。后端消费者再按照 MySQL 能承受的节奏慢慢拉取消息、异步落库发券，彻底保护数据库不被打垮。"

📌 **记忆骨架（上场前脑子里过这 4 个词就行）：**

1. **前端：** 按钮置灰 + 验证码（防手抖、打散峰值）
2. **网关（Gateway + Sentinel）：** 限 IP/UserID（防刷） + 令牌桶全局限流（10万拦成3000）
3. **业务（Redis + 本地内存）：** Lua 脚本原子预扣减 + JVM 内存售罄标记（保 Redis、防超卖）
4. **数据（MQ + MySQL）：** 1000个成功请求走 MQ 异步落库（削峰填谷保 DB）